AI FITTRACK – EASY RUN SOURCE CODE

Simple full-stack college project source-code guide.

Technology: Python Flask + SQLite + HTML/CSS/JavaScript.

How to run: install Python, create the files shown in this report, run python app.py, then open http://127.0.0.1:5000 in a browser.

PROJECT OVERVIEW

AI FitTrack is a simple fitness tracking web application. Users can enter their name, fitness goal, workout and duration. The backend stores the data in SQLite and generates a basic recommendation.

Project structure:

```
AI_FitTrack/
  app.py
  templates/index.html
  static/style.css
  fitness.db (created automatically)
```

REQUIREMENTS

Install Python 3.10 or newer. Flask is the only external Python package required.

Install Flask using: pip install flask.

No separate MySQL server is required because SQLite is included with Python.

BACKEND – app.py

Create app.py with the following code:

BACKEND CODE – PART 1

```
from flask import Flask, render_template, request, redirect
import sqlite3

app = Flask(__name__)

def db():
    con = sqlite3.connect("fitness.db")
    con.row_factory = sqlite3.Row
    return con

def init_db():
    con = db()
    con.execute("""CREATE TABLE IF NOT EXISTS workouts(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT, goal TEXT, workout TEXT,
        duration INTEGER, recommendation TEXT)""")
    con.commit()
    con.close()

def recommend(goal, workout):
    if goal == "Weight Loss":
        return "Try regular cardio and maintain a balanced diet."
    if goal == "Muscle Gain":
        return "Include strength training and adequate protein."
    return "Maintain a regular workout routine and stay hydrated."
```

BACKEND CODE – PART 2

```
@app.route("/")
def home():
    con = db()
    rows = con.execute("SELECT * FROM workouts ORDER BY id DESC").fetchall()
    con.close()
    return render_template("index.html", workouts=rows)

@app.route("/add", methods=["POST"])
def add():
    name = request.form["name"]
    goal = request.form["goal"]
    workout = request.form["workout"]
    duration = int(request.form["duration"])
    rec = recommend(goal, workout)

    con = db()
    con.execute(
        "INSERT INTO workouts(name,goal,workout,duration,recommendation) VALUES(?,?,?,?,?)",
        (name, goal, workout, duration, rec))
    con.commit()
    con.close()
    return redirect("/")

if __name__ == "__main__":
    init_db()
    app.run(debug=True)
```

FRONTEND – index.html

Create a folder named templates, then create templates/index.html:

HTML CODE

```
<!DOCTYPE html>
<html>
<head>
  <title>AI FitTrack</title>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body>
<div class="container">
  <h1>AI FitTrack</h1>
  <p>Simple AI Fitness Tracking System</p>

  <form action="/add" method="post">
    <input name="name" placeholder="Your Name" required>
    <select name="goal">
      <option>Weight Loss</option>
      <option>Muscle Gain</option>
      <option>General Fitness</option>
    </select>
    <input name="workout" placeholder="Workout name" required>
    <input name="duration" type="number" placeholder="Duration (minutes)" required>
    <button type="submit">Add Workout</button>
  </form>

  <h2>Workout History</h2>
  {% for w in workouts %}
    <div class="card">
      <b>{{ w['name'] }}</b> — {{ w['workout'] }}
      <br>Goal: {{ w['goal'] }}
      <br>Duration: {{ w['duration'] }} minutes
      <br><strong>AI Suggestion:</strong> {{ w['recommendation'] }}
    </div>
  {% endfor %}
</div>
</body>
</html>
```

CSS – style.css

Create a folder named static, then create static/style.css:

CSS CODE

```
body {
  font-family: Arial, sans-serif;
  background: #f2f5f7;
  margin: 0;
}
.container {
  max-width: 800px;
  margin: 40px auto;
  background: white;
  padding: 25px;
  border-radius: 12px;
}
input, select, button {
  width: 100%;
  padding: 12px;
  margin: 7px 0;
  box-sizing: border-box;
}
button {
  cursor: pointer;
}
.card {
  padding: 15px;
  margin: 10px 0;
  border: 1px solid #ddd;
  border-radius: 8px;
}
```

HOW TO RUN – WINDOWS

1. Install Python.
2. Create the project folders and files.
3. Open Command Prompt inside the AI_FitTrack folder.
4. Run `pip install flask`.
5. Run `python app.py`.
6. Open `http://127.0.0.1:5000` in Chrome.

HOW TO RUN – VS CODE

Open the AI_FitTrack folder in VS Code. Open Terminal → New Terminal. Run pip install flask, then python app.py. When Flask starts, open the local address shown in the terminal.

Stop the server with Ctrl+C.

DATABASE & AI LOGIC

The SQLite database is automatically created as fitness.db when the application starts. Each workout is stored with the user's name, goal, workout, duration and recommendation.

The recommendation function is a simple rule-based AI demonstration. For a college prototype, it is easy to understand and easy to extend with machine-learning models later.

PROJECT FLOW & TESTING

Flow: User opens website → enters fitness details → submits form → Flask receives data → recommendation function runs → SQLite stores record → dashboard displays workout and suggestion.

Test cases: submit valid workout; check that it appears in history; test each goal option; restart the server and confirm stored records remain in SQLite.

FINAL NOTES / VIVA

Frontend: HTML, CSS and Jinja template syntax. Backend: Python Flask. Database: SQLite. AI: rule-based personalized recommendation.

Future improvements: login system, charts, calorie tracking, wearable integration, nutrition module and a trained ML recommendation model.

This source is intentionally simple so it can be run and explained easily for a college project.